Week-1 Report

Problem-1

race_counter.c

- For this problem, we expected that the output will be 2 * 1e7 but the actual output is almost always different with a very large difference.
- When I changed the ITERS value to 1000 then I got the expected answer (2000) in several runs.
- When I changed the ITERS value to 1e8 it was almost the same as the original meaning the values varied from the expected answer.
- When I compiled the code with -O2 included then the output always matched the expected output (in all the 10 cases I ran at least).

parallel_sum.c

- The final output is consistently not even half of the actual result and when the number of threads it seems that answer gets closer and closer to actual answer (in most of the cases).

bank_chaos.c

- The balance is always zero at the end but the total successful attempts of Alice and Bob taking out money is (almost always) greater than the total balance which shows the race condition.

Week-2 Report

Problem -1:

parallel_sum_fixed.c

When timing the fixed version of the code it consistently performs slower than the normal version. This is due to fact that the threads have to wait for locks to be released in the fixed version and this extra ‘wait time’ causes the program to be a slower (although just a difference of 0.002-0.003 seconds)

Week-3 Report

Problem-1:

parallel_statistics.c

Threads compute their slice statistics and pack them into a heap-allocated worker_result_t structure. This structure pointer is passed through pthread_exit() and safely gathered by the main thread during pthread_join(), completely eliminating memory safety risks from dangling stack pointers.

Problem-2:

sem_buffer.c

In the condition variable pattern, threads check conditions within a while loop because signals can be spurious. Semaphores, however, inherently maintain an internal integer counter. Since a semaphore automatically decrements or increments atomic counts natively, threads sleep and wake up without requiring manual condition check loops.

Week-4 Report

Problem-1: